**MVP Implementation Plan — Personal Dark-Mode Productivity Tool**

**Tech Stack Recap**

- **Frontend:** React + TypeScript + Tailwind CSS + shadcn/ui
- **Backend:** FastAPI + SQLAlchemy + SQLite
- **Optional Utilities:** Prism.js (snippets), react-hotkeys-hook (keyboard shortcuts), Day.js (dates)
- **Local Hosting:** localhost, optional Docker Compose

---

**Phase 0 — Project Setup**

1. Initialize **monorepo**:

/my-productivity-app

├─ /frontend (Next.js + TypeScript)

├─ /backend (FastAPI + SQLAlchemy)

└─ /db (local SQLite file)

2. Install dependencies:

**Frontend**

npm create next-app frontend --ts

cd frontend

npm install tailwindcss shadcn-ui @dnd-kit/core react-markdown react-hotkeys-hook prismjs dayjs lucide-react

**Backend**

python -m venv venv

source venv/bin/activate

pip install fastapi uvicorn sqlalchemy pydantic sqlite3

3. Configure **Tailwind** for dark mode:

// tailwind.config.js

```
module.exports = {
  darkMode: 'class',
  content: ['./pages/**/*.{js,ts,jsx,tsx}', './components/**/*.{js,ts,jsx,tsx}'],
  theme: { extend: {} },
  plugins: [],
}
```

---

**Phase 1 — Core Daily Log MVP**

**Goal:** Daily log editor + dark mode + cursor focus

**Backend**

- Table: daily_logs
    - id (int, PK)
    - date (YYYY-MM-DD)
    - content (TEXT)
    - created_at, updated_at
- Endpoints:
    - GET /logs/today
    - POST /logs (create/update today's log)
    - GET /logs/:date (optional for navigation)

**Frontend**

- Home screen:
    - Header: Today's date
    - Daily log editor: free-text, cursor focused
    - Auto-save every few seconds
    - Markdown rendering
- Dark mode applied across entire content area

- Keyboard: Cmd + D → focus today

---

**Phase 2 — Tasks MVP**

**Goal:** Minimal tasks alongside logs

**Backend**

- Table: tasks
    - id, title, status (Not Started / In Progress / Done), created_at, updated_at
- Optional link to log lines (later)

**Frontend**

- Section under daily log: **In Progress Tasks**
- Ability to add tasks quickly (+ Task button)
- Click task → side panel shows details
- Status badges in amber (in-progress) or green (done)
- Minimal hover effect

---

**Phase 3 — Notes, Snippets, Bookmarks MVP**

**Goal:** Capture context beyond tasks

**Backend**

- Table: notes (id, title, content, created_at)
- Table: snippets (id, title, language, code, created_at)
- Table: bookmarks (id, title, url, created_at)

**Frontend**

- Sidebar navigation
- Quick add buttons
- List of recent items under Home (optional)

- Inline linking from log lines (@ triggers search to link tasks/notes/snippets/bookmarks)

---

**Phase 4 — Search & Linking**

**Goal:** Quickly find anything and create links

**Backend**

- Full-text search on logs, tasks, notes, snippets, bookmarks (SQLite FTS)

**Frontend**

- Global search (Cmd+K)
- Inline linking via @
- Side panel shows related tasks/notes/snippets/bookmarks
- Optional hover preview of linked item

---

**Phase 5 — Polishing & Dark Mode UI**

- Refine spacing, font sizes, hover effects
- Sidebar active indicator
- Cards with subtle shadow / elevation
- Accent colors: blue for actions, amber for in-progress, green for done
- Keyboard-first shortcuts implemented
- Auto-save and undo in Daily Log

---

**Phase 6 — Optional Enhancements**

- Markdown + code snippet syntax highlighting (Prism.js)
- Undo/redo in Daily Log
- Export Daily Logs (PDF/Markdown)
- Export/backup SQLite DB

- Drag-and-drop ordering of tasks (via @dnd-kit/core)

---

**Phase 7 — Future-Proofing**

- Easy migration to Postgres / cloud backend
- Multi-user support
- Desktop app wrapper (Tauri)
- AI-assisted suggestions / auto-linking

---

**Immediate Next Steps for You**

1. Setup repo with frontend + backend
2. Build **Phase 1 — Daily Log MVP** (dark-mode editor + cursor focus)
3. Implement **Phase 2 — Tasks MVP** alongside the log
4. Wire up SQLite locally via SQLAlchemy
5. Connect frontend to backend via API calls
6. Test daily log editor + task list in dark mode

**SQLite + SQLAlchemy Schema**

---

### 1️⃣ daily_logs

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | Auto-increment |
| date | DATE | Unique — one log per day |
| content | TEXT | Free-text Markdown |
| created_at | DATETIME | Auto timestamp |
| updated_at | DATETIME | Auto timestamp on edit |

```
class DailyLog(Base):
  __tablename__ = "daily_logs"
  id = Column(Integer, primary_key=True)
  date = Column(Date, unique=True, nullable=False)
  content = Column(Text, default="")
  created_at = Column(DateTime, default=datetime.utcnow)
  updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

  tasks = relationship("Task", back_populates="log")
```

---

### 2️⃣ tasks

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| title | TEXT | Task title |
| status | TEXT | “Not Started”, “In Progress”, “Done” |

| Column | Type | Notes |
| --- | --- | --- |
| log_id | INTEGER FK | Optional link to daily_log |
| created_at | DATETIME | |
| updated_at | DATETIME | |

```
class Task(Base):
  __tablename__ = "tasks"
  id = Column(Integer, primary_key=True)
  title = Column(Text, nullable=False)
  status = Column(String(20), default="Not Started")
  log_id = Column(Integer, ForeignKey("daily_logs.id"), nullable=True)
  created_at = Column(DateTime, default=datetime.utcnow)
  updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

  log = relationship("DailyLog", back_populates="tasks")
```

---

### 3️⃣ notes

| Column | Type | Notes |
| --- | --- | --- |
| id | INTEGER PK | |
| title | TEXT | Optional short title |
| content | TEXT | Markdown / plain text |
| created_at | DATETIME | |
| updated_at | DATETIME | |

```
class Note(Base):
  __tablename__ = "notes"
```

```
    id = Column(Integer, primary_key=True)
    title = Column(Text, nullable=True)
    content = Column(Text, default="")
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

---

### 4️⃣ snippets

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| title | TEXT | Short description |
| language | TEXT | e.g., SQL, Python, Bash |
| code | TEXT | |
| created_at | DATETIME | |
| updated_at | DATETIME | |

```
class Snippet(Base):
    __tablename__ = "snippets"
    id = Column(Integer, primary_key=True)
    title = Column(Text, nullable=False)
    language = Column(Text, nullable=False)
    code = Column(Text, default="")
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

---

### 5️⃣ bookmarks

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| title | TEXT | Short name for link |
| url | TEXT | |
| created_at | DATETIME | |
| updated_at | DATETIME | |

```
class Bookmark(Base):
    __tablename__ = "bookmarks"
    id = Column(Integer, primary_key=True)
    title = Column(Text, nullable=False)
    url = Column(Text, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

---

### 6️⃣ Optional Linking Table (for many-to-many connections)

For **linking logs ↔ tasks ↔ notes ↔ snippets ↔ bookmarks** without forcing structure:

```
class LogTaskLink(Base):
    __tablename__ = "log_task_link"
    id = Column(Integer, primary_key=True)
    log_id = Column(Integer, ForeignKey("daily_logs.id"))
    task_id = Column(Integer, ForeignKey("tasks.id"))


class NoteTaskLink(Base):
    __tablename__ = "note_task_link"
    id = Column(Integer, primary_key=True)
```

```
    note_id = Column(Integer, ForeignKey("notes.id"))
    task_id = Column(Integer, ForeignKey("tasks.id"))

# Similar tables can be added for snippets & bookmarks if needed
```

✅ Notes:

- These links are **optional**; your daily log doesn't require tasks.
- This is **future-proof** if you want to show "all logs / notes referencing this task" later.

---

### 7️⃣ SQLite Setup in FastAPI

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

SQLALCHEMY_DATABASE_URL = "sqlite:///./db/local.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()
```

- local.db is a single file in /db/
- Easy to backup or move
- Supports all relationships we need for MVP